

ConstRSA: A Minimal Constant-Time RSA-512 Digital Signature with Statistical Timing-Leak Verification on Standard Laptops

Maleesha Rathnayake
2020/ICT/47

University of Vavuniya

January 2026

Outline

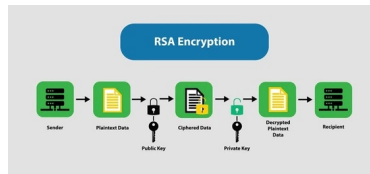
- Introduction
- Problem Definition
- Literature Review
- Research Gap
- Objective
- Methodology
- Expected Outcome
- Timeline
- References

Introduction

Digital signatures prove who created a message and confirm that it was not changed after signing. A signer uses a **private key** to create the signature and a verifier uses the **public key** to verify it.

Digital signatures are used in:

- software updates
- secure communication
- online banking
- document authentication



Common signature algorithms: **RSA, DSA, ECDSA**.

Real implementations can leak information when execution time or CPU behavior depends on secret data.

Problem Definition

- No small and easy-to-understand RSA signature algorithm that is clearly constant-time and testable on a normal laptop.
- Constant-time means the program takes almost the same time even with different secret key values.
- Attackers can measure timing differences and guess parts of the private key.
- Existing RSA implementations are inside large libraries and are hard for learning and benchmarking.
- Even small timing/cache/CPU differences can be dangerous if many measurements are collected.

Literature Review (Summary Table)

Work	Context	Focus	Key Point
Luo et al. (TACO)	GPU RSA	Timing leakage	Optimizations do not guarantee constant-time behavior.
Luo et al. (ICCAD)	GPU RSA	Timing attack	Platform-specific choices can introduce measurable leakage.
Kou et al. (DATE)	Secure environments	Cache leakage	Secret-dependent memory access can leak exponent information.
Aldaya & Brumley	CPU behavior	Side-channels	Small leakage can still enable key recovery with many traces.
Doctoral thesis	Survey / background	Impl. security	Side-channel concepts and evaluation guidance for practice.
Ding	RSA signatures	Overview	RSA signature background for motivation and context.

Takeaway: Constant-time coding + timing tests are needed to confirm low leakage.

- Side-channel leakage is still practical on modern platforms.
- Constant-time engineering is needed, but most implementations are large and complex.
- Lack of a compact, auditable, reproducible RSA signature implementation suitable for:
 - undergraduate learning,
 - laptop experiments,
 - timing-leak evaluation.

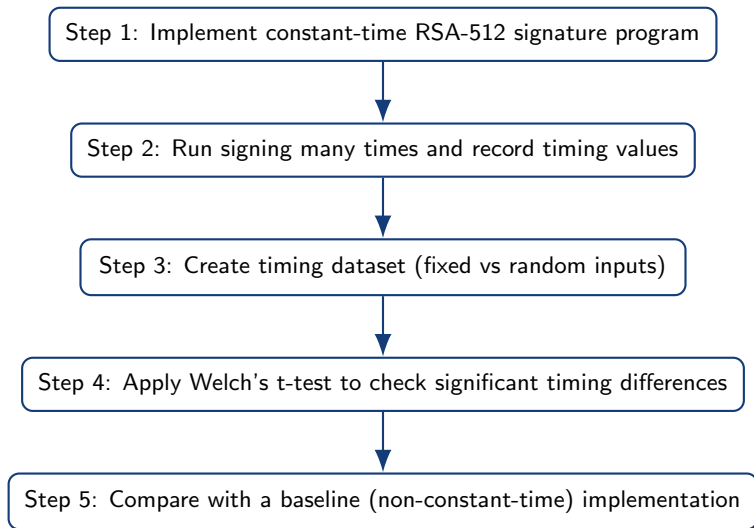
Objective

Primary Objective

Design, implement, and statistically validate a minimal **constant-time RSA-512 digital signature** (RSA-PSS + SHA-256) under **10 KB**, suitable for standard laptops.

Specific Objectives (Roadmap)

- 1 Implement branchless RSA-512 signing in C (CRT).
- 2 Remove secret-dependent branching and memory access patterns.
- 3 Collect timing traces and apply Welch's t-test for leakage testing.
- 4 Benchmark signing and verification performance.
- 5 Publish implementation and scripts as open-source.



Expected Outcome



- 1 Minimal constant-time RSA-512 signature implementation (target: under 10 KB)
- 2 Statistical results from timing experiments (detectable leakage or not)
- 3 Open-source code + documentation + testing scripts
- 4 Clear example for secure coding and cryptography education

Timeline (Gantt Chart)

Work/Time	Nov-25	Dec-25	Jan-26	Feb-26	Mar-26	Apr-26	May-26	Jun-26
Title selection								
Proposal								
Literature Review								
Resource gathering								
Tools and Techniques								
Implementation								
Writing								
Publication / Conferences								
Presentation and Submission								

References



Luo et al., TACO 2019, DOI: 10.1145/3341729.



Luo et al., ICCAD 2018, DOI: 10.1145/3240765.3240812.



Kou et al., DATE 2023, DOI: 10.23919/DATE56975.2023.10137116.



Doctoral thesis, is.muni.cz/th/rbeu1/doctoral_thesis.pdf.



Aldaya & Brumley, TCHES 2020, DOI: 10.13154/tches.v2020.i2.196-221.



Ding, 2024, DOI: 10.62051/j2p6g984.

Thank You

Thank you.
Questions?